Recherche de zéros et systèmes non-linéaires

Vincent Nozick

IMAC INGÉNIEUR ESIPE

Université Gustave Eiffel

Recherche de zéros

Problématique :

Étant donnée une fonction non-linéaire, on veut trouver les points où cette fonction s'annule, s'ils existent : ses *zéros* (aussi appelés *racines*)

Recherche de zéros

Fonctions étudiées :

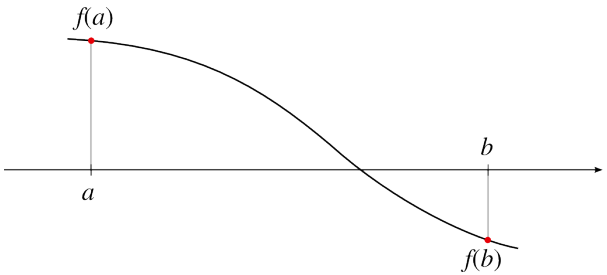
- fonctions analytiques
  (fonctions dont on connait la formule)
  $\mathbb{R} \rightarrow \mathbb{R}$, continues et dérivables
- fonctions non analytiques, mais évaluables en n'importe quel point.
- un ensemble de points représentant une fonction supposée continue.

Recherche de zéros



On peut évaluer cette fonction sur quelques points,
mais on ne sait pas à quoi ressemble la courbe.

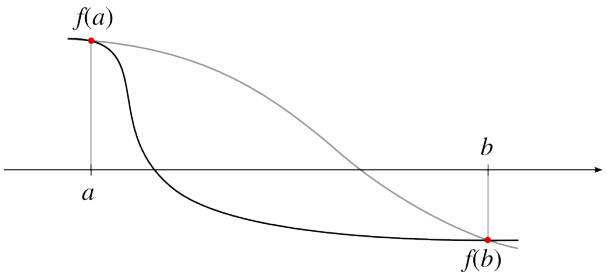
Recherche de zéros



On peut évaluer cette fonction sur quelques points,
mais on ne sait pas à quoi ressemble la courbe.

Méthode naïve Dichotomie Méthode de la fausse position Newton n-dimensions Levenberg-Marquardt Applications

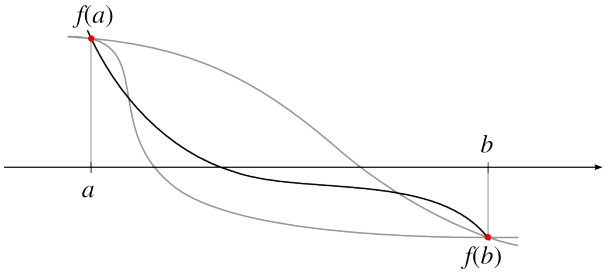
Recherche de zéros



On peut évaluer cette fonction sur quelques points,
mais on ne sait pas à quoi ressemble la courbe.

Méthode naïve Dichotomie Méthode de la fausse position Newton n-dimensions Levenberg-Marquardt Applications

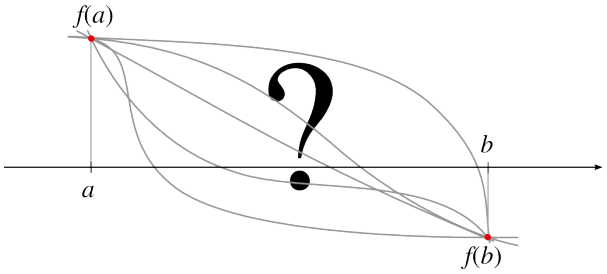
Recherche de zéros



On peut évaluer cette fonction sur quelques points,
mais on ne sait pas à quoi ressemble la courbe.

Méthode naïve Dichotomie Méthode de la fausse position Newton n-dimensions Levenberg-Marquardt Applications

Recherche de zéros



On peut évaluer cette fonction sur quelques points,
mais on ne sait pas à quoi ressemble la courbe.

Méthode naïve

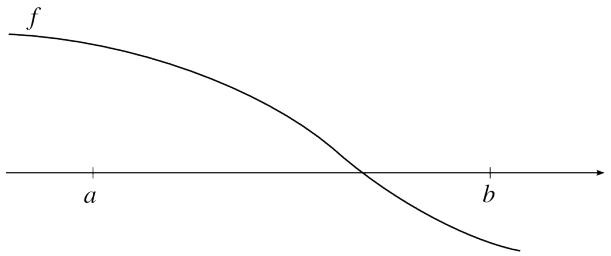
Soit f une fonction :

- continue sur $[a, b]$
- monotone sur $[a, b]$
- telle que $f(a)f(b) < 0$

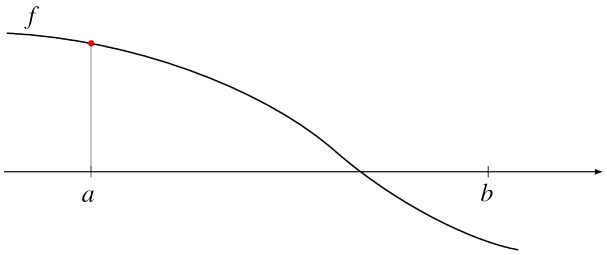
Méthode :

- on choisit un pas dx
- on teste $f(a + k.dx)$ jusqu'à ce que f change de signe (k entier positif)

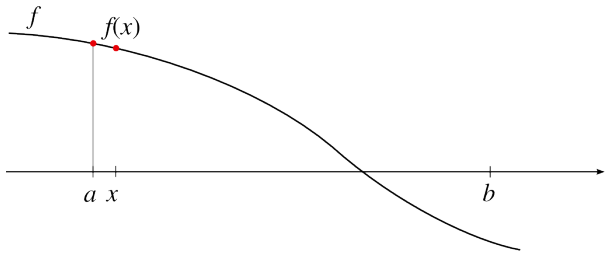
Méthode naïve



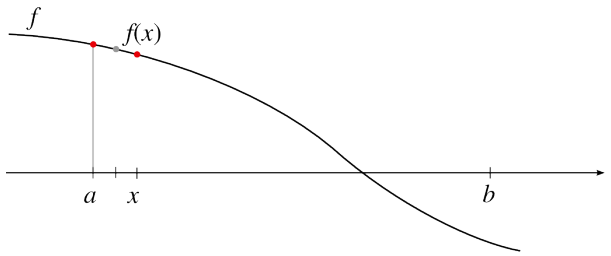
Méthode naïve



Méthode naïve

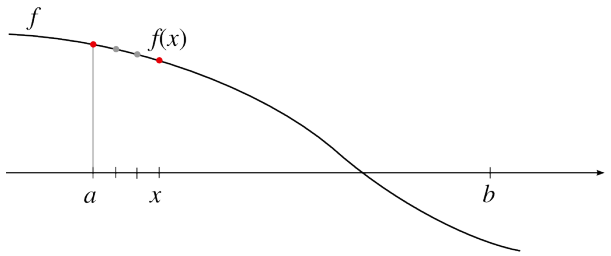


Méthode naïve



Méthode naïve Dichotomie Méthode de la fausse position Newton n-dimensions Levenberg-Marquardt Applications

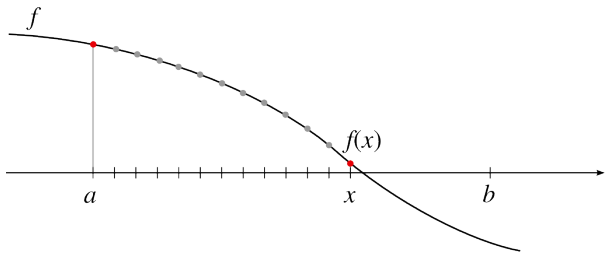
Méthode naïve



Vincent Nozick Recherche de zéros et systèmes non-linéaires 6 / 52

Méthode naïve Dichotomie Méthode de la fausse position Newton n-dimensions Levenberg-Marquardt Applications

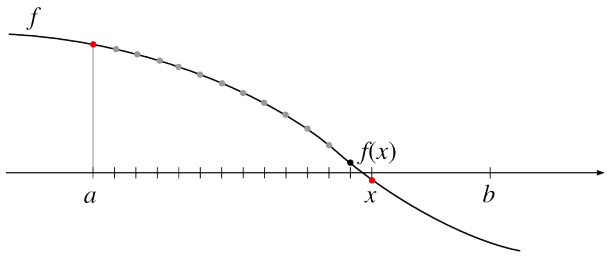
Méthode naïve



Vincent Nozick Recherche de zéros et systèmes non-linéaires 6 / 52

Méthode naïve Dichotomie Méthode de la fausse position Newton n-dimensions Levenberg-Marquardt Applications

Méthode naïve



Vincent Nozick Recherche de zéros et systèmes non-linéaires 6 / 52

Méthode naïve

Algorithm 1: méthode naïve

input: une fonction f et un intervalle $[a, b]$

```
x = a + dx
while f(x)f(a) > 0 do
    x = x + dx
return [x − dx, x]
```

Remarque :

- Le résultat est l'intervalle $[x - dx, x]$.
- On peut relancer le même algo sur cet intervalle avec un pas dx plus petit.

Méthode naïve

Réflexion :

C'est nul comme méthode.

Dichotomie

Soit f une fonction :

- continue sur $[a, b]$
- monotone sur $[a, b]$
- telle que $f(a)f(b) < 0$

Méthode :
On recherche un petit intervalle contenant le zéro.

Dichotomie

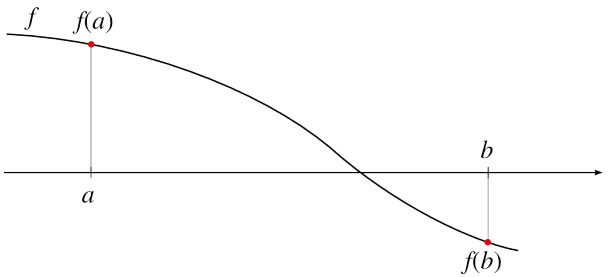
Soit f une fonction :

- continue sur $[a, b]$
- monotone sur $[a, b]$
- telle que $f(a)f(b) < 0$

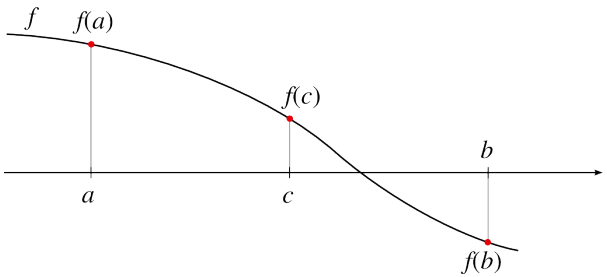
Méthode :
On recherche un petit intervalle contenant le zéro.

- on coupe l'intervalle de départ en 2
- on garde l'intervalle qui contient le 0
- on recommence ...

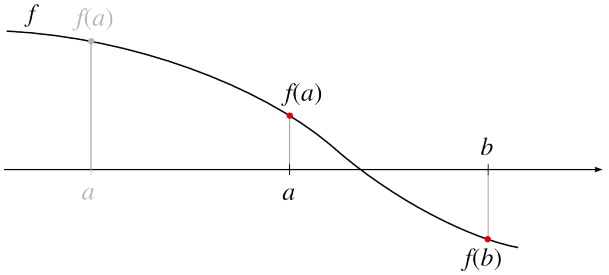
Dichotomie



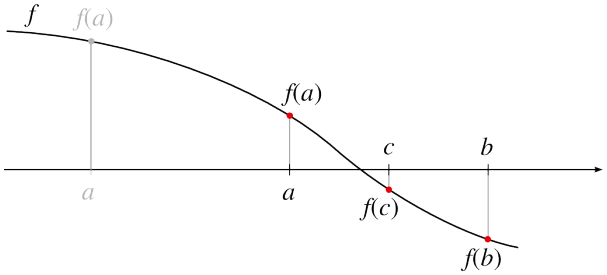
Dichotomie



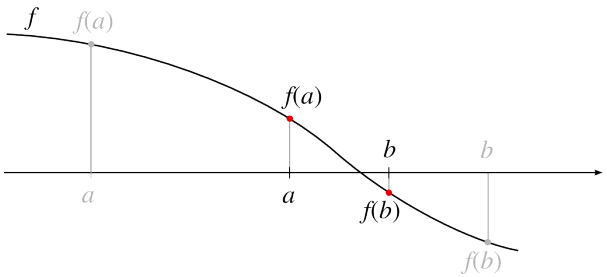
Dichotomie



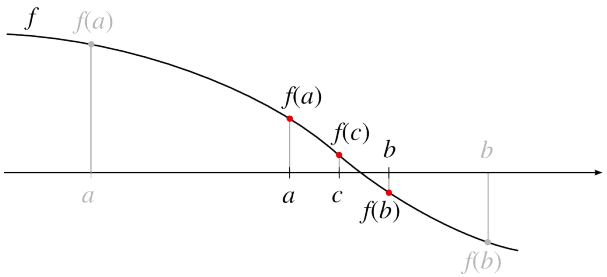
Dichotomie



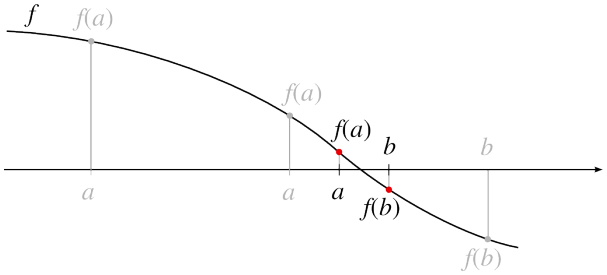
Dichotomie



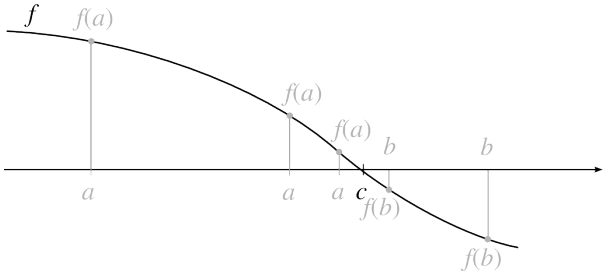
Dichotomie



Dichotomie



Dichotomie



Dichotomie

Algorithm 2: Dichotomie

input: fonction f, intervalle $[a, b]$ et nombre d'itérations n

```
repeat n times
    c = (a + b)/2
    if f(a)f(c) < 0 then
        b = c
    else
        a = c
return [a, b]
```

Remarque :
Pour obtenir une précision 2 fois supérieure, il suffit de faire une itération de plus.

Dichotomie

Réflexions :

- La dichotomie ne fait appel qu'au signe, pas aux valeurs de la fonction.
- La dichotomie est une amélioration de la méthode naïve récursive, où le pas vaut la moitié de l'intervalle étudié.

Méthode de la fausse position

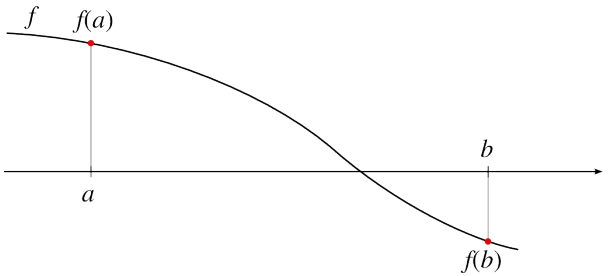
Soit f une fonction :

- continue sur $[a, b]$
- monotone sur $[a, b]$
- telle que $f(a)f(b) < 0$

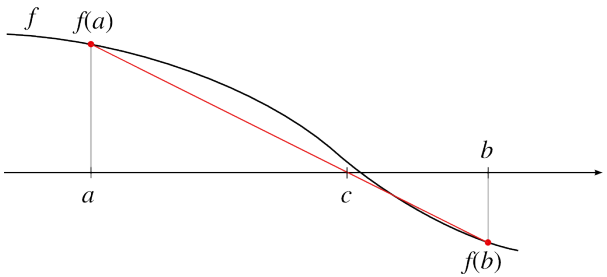
Méthode :

- on fait une approximation linéaire de f sur $[a, b]$
  - on approxime la courbe de f par sa *corde*
- on résout l'équation linéaire
  - on calcule l'équation de la droite passant par $(a, f(a))$, $(b, f(b))$
  - on calcule son intersection avec l'axe des abscisses
- on obtient un nouvel intervalle
- on recommence

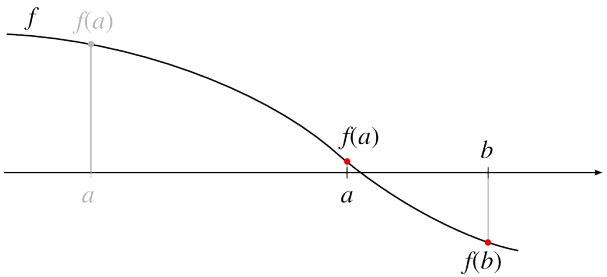
Méthode de la fausse position



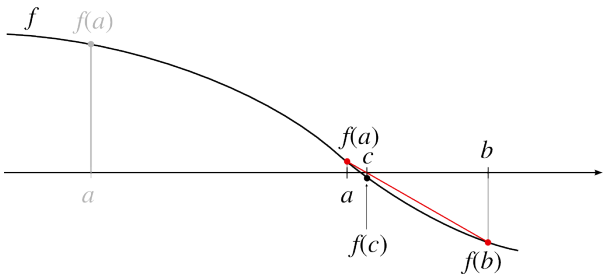
Méthode de la fausse position



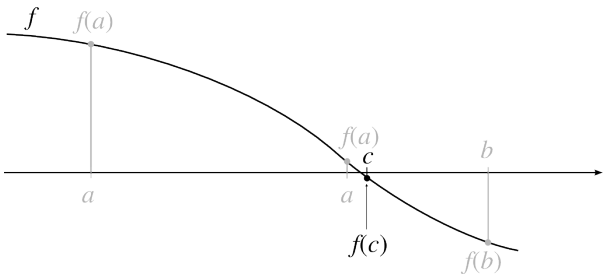
Méthode de la fausse position



Méthode de la fausse position



Méthode de la fausse position



Méthode de la fausse position

$$y = m.x + p = \frac{f(b) - f(a)}{b - a}x + p$$
$$p = f(a) - \frac{f(b) - f(a)}{b - a}a$$
$$y = \frac{f(b) - f(a)}{b - a}x + f(a) - \frac{f(b) - f(a)}{b - a}a$$
$$0 = \frac{f(b) - f(a)}{b - a}c + f(a) - \frac{f(b) - f(a)}{b - a}a$$

$$c = \frac{a.f(b) - b.f(a)}{f(b) - f(a)} = ... = a + \frac{f(a)(b - a)}{f(a) - f(b)}$$

Méthode de la fausse position

Algorithm 3: Fausse position

input: fonction f, intervalle $[a, b]$ et un *seuil*

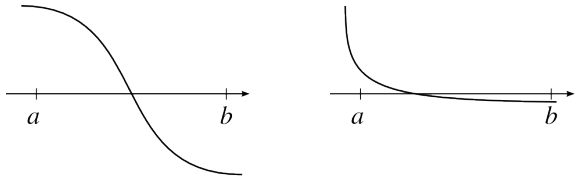
```
repeat
    c = a + f(a)(b − a) / (f(a) − f(b))
    if f(a)f(c) < 0 then
        b = c
    else
        a = c
until |f(c)| < seuil
return c
```

on s'arrête quand $|f(c)|$ est suffisamment petit

Méthode de la fausse position

Réflexion :

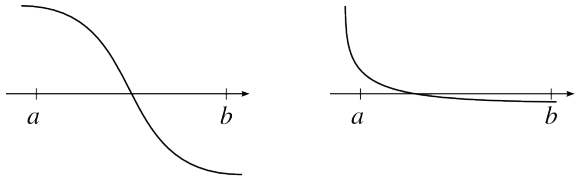
Cette méthode peut être inadaptée pour certaines fonctions.



Méthode de la fausse position

Réflexion :

Cette méthode peut être inadaptée pour certaines fonctions.



OK **PAS OK**

Méthode de la fausse position

Réflexion :

La méthode de la fausse position est adaptée aux fonction dont le comportement est à peu près linéaire sur $[a, b]$, ce qui permet de restreindre $[a, b]$ plus rapidement qu'avec la méthode de dichotomie.

Elle fait appel aux valeurs de f (contrairement à la méthode par dichotomie).

Newton

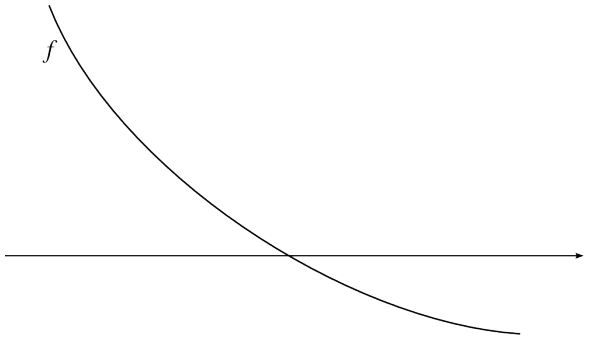
Principe :

- hypothèse : x_0 proche du résultat
- méthode itérative

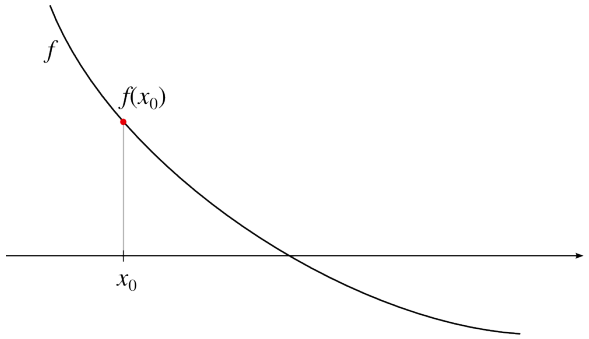
Méthode :

- on calcule la tangente au point $\big(x_i, f(x_i)\big)$
- on calcule l'intersection de la tangente avec l'axe des abscisses
- on obtient x_{i+1}
- on recommence

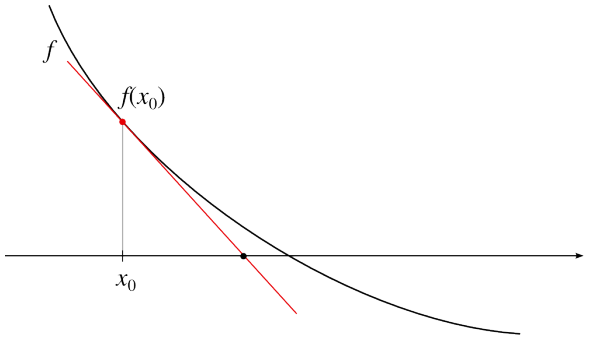
Newton



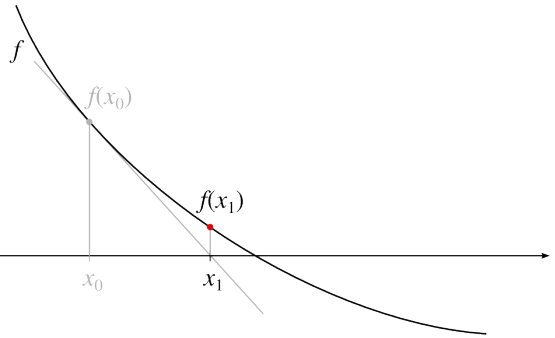
Newton



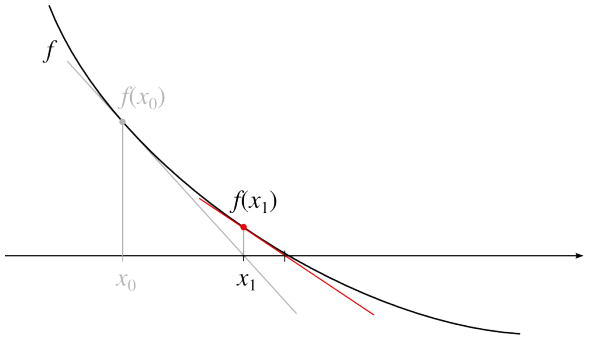
Newton



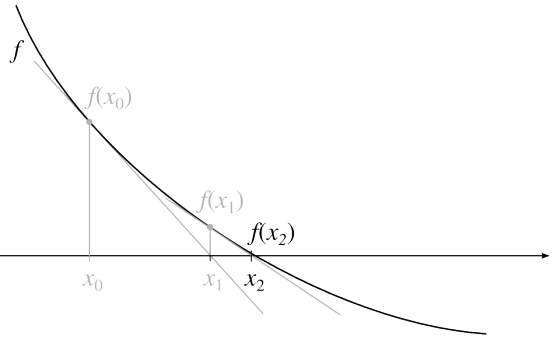
Newton



Newton



Newton



Newton

Tangente ...

- tangente : $y = mx + p$

Newton

Tangente ...

- tangente : $y = mx + p$

- dérivée de f en $x_0 \rightarrow$ coefficient directeur de la tangente

$\rightarrow$ tangente : $y = f'(x_0)x + p$

Newton

Tangente ...

- tangente : $y = mx + p$

- dérivée de f en $x_0 \rightarrow$ coefficient directeur de la tangente

$\rightarrow$ tangente : $y = f'(x_0)x + p$

- avec $p = y_0 - f'(x_0)x_0$

$\rightarrow$ tangente : $y = f'(x_0)x + y_0 - f'(x_0)x_0$

Newton

Intersection avec les abscisses ...

- tangente : $y = f'(x_0)x + y_0 - f'(x_0)x_0$

- si $y_1 = 0$ (et $y_0 = f(x_0)$)

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$

Newton

Algorithm 4: Newton

input: fonction f, solution initiale x_0 et un *seuil*

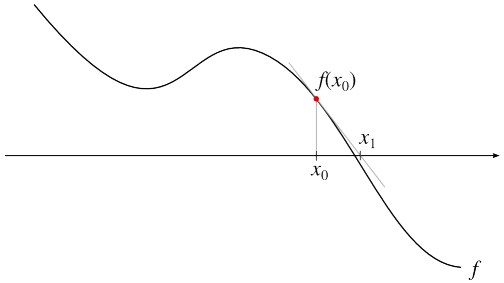
1 $x = x_0$
2 **repeat**
3 $\quad x = x - f(x)/f'(x)$
4 **until** $|f(x)| < seuil$
5 **return** x

On s'arrête quand $|f(x)|$ est suffisamment petit

Newton

Remarque :

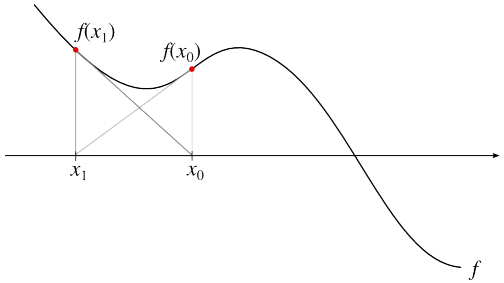
Selon les conditions initiales ...



Newton

Remarque :

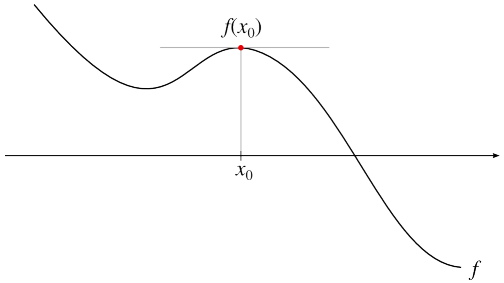
Selon les conditions initiales ...



Newton

Remarque :

Selon les conditions initiales ...



Newton

Réflexion :

- le choix de la position initial est très important
- la convergence n'est pas garantie
  (contrairement aux autres méthodes)

mais ...

- la convergence peut être très rapide

Méthode de la sécante

Méthode :

On veut utiliser la méthode de Newton, mais on ne connait pas la dérivée f' de f.

Méthode de la sécante

Méthode :

On veut utiliser la méthode de Newton, mais on ne connait pas la dérivée f' de f.

$\rightarrow$ on utilise les nombres duaux.

Méthode de la sécante

Méthode :

On veut utiliser la méthode de Newton, mais on ne connait pas la dérivée f' de f.

$\rightarrow$ on utilise les nombres duaux.

$\rightarrow$ on fait une estimation de la dérivée de f en x_0 qu'on réinjecte dans la méthode de Newton.

Méthode de la sécante

Dérivée :

$$f'(x_0) = \lim_{x \to x_0} \frac{f(x) - f(x_0)}{x - x_0}$$

Estimation numérique de la dérivée (dérivée arrière) :

$$f'(x) \simeq \frac{f(x) - f(x - \Delta x)}{\Delta x}$$

Méthode de la sécante

Méthode :

- on évalue la dérivée de $f(x_i)$
- on calcule la tangente au point $(x_i, f(x_i))$
- on calcule l'intersection de la tangente avec l'axe des abscisses
- on obtient x_{i+1}
- on recommence ....

Méthode de la sécante

Itérations :

$$\begin{aligned} x_{n+1} &= x_n - \Delta x \frac{f(x_n)}{f(x_n) - f(x_n - \Delta x)} \\ &= \frac{(x_n - \Delta x) f(x_n) - x_n f(x_n - \Delta x)}{f(x_n) - f(x_n - \Delta x)} \end{aligned}$$

Méthode de la sécante

Algorithm 5: Newton - estimation de la dérivée

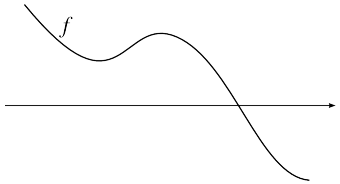
input: fonction f, solution initiale x_0, un pas Δx et un *seuil*

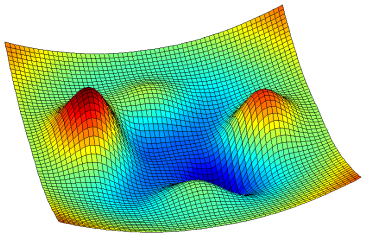
1 $x = x_0$
2 **repeat**
3 $\quad x = \frac{f(x)(x-\Delta x) - x.f(x-\Delta x)}{f(x) - f(x-\Delta x)}$
4 **until** $|f(x)| <$ *seuil*
5 **return** x

On s'arrête quand $|f(x)|$ est suffisamment petit

Systèmes non-linéaires

Et pour les fonctions de plusieurs variables?



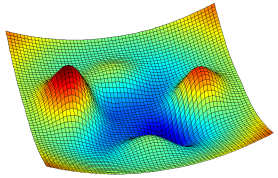


$z = f(x)$
1D

$\mathbf{z} = f(x, y)$
2D

Systèmes non-linéaires

Certaines fonctions ont des images vectorielles :



$$\mathbf{z} = f(\mathbf{x})$$

$$\text{avec } \mathbf{z} = \begin{pmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{pmatrix} \text{ et } \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

Zéros d'une fonction de plusieurs variables

Problème de minimisation :

soit la fonction $\mathbf{z} = f(\mathbf{x})$
on cherche $\widehat{\mathbf{x}}$ tel que $f(\widehat{\mathbf{x}}) = \widehat{\mathbf{z}}$, soit $||f(\widehat{\mathbf{x}}) - \widehat{\mathbf{z}}|| = 0$.

→ on veut minimiser $\epsilon = \|f(\mathbf{x}) - \widehat{\mathbf{z}}\|$
autrement dit, on veut faire tendre $f(\mathbf{x})$ vers $\widehat{\mathbf{z}}$ autant que possible.

→ on part d'une estimation $\mathbf{x}_0$

Analogie avec le cas unidimensionnel

Dérivée 1D :

$$f'(x) \simeq \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

Soit

$$f(x + \Delta x) \simeq f(x) + f'(x)\Delta x$$

Dérivée en n dimensions :

$$f(\mathbf{x} + \Delta_{\mathbf{x}}) \simeq f(\mathbf{x}) + \mathbf{J}(\mathbf{x})\Delta_{\mathbf{x}}$$

où $\mathbf{J}(\mathbf{x})$ est la matrice jacobienne de f en $\mathbf{x}$.

Optimisation non-linéaire

Soit une fonction f à plusieurs variables de $\mathbb{R}^n \mapsto \mathbb{R}^m$:

$$f : \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \longmapsto \begin{pmatrix} f_1(x_1, x_2, \cdots, x_n) \\ f_2(x_1, x_2, \cdots, x_n) \\ \vdots \\ f_m(x_1, x_2, \cdots, x_n) \end{pmatrix}$$

Définition:
La dérivée partielle de f est la dérivée par rapport à l'une de ses variables, les autres étant gardées constantes.

Optimisation non-linéaire

Soit une fonction f à plusieurs variables de $\mathbb{R}^n \mapsto \mathbb{R}^m$:

$$f : \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \longmapsto \begin{pmatrix} f_1(x_1, x_2, \cdots, x_n) \\ f_2(x_1, x_2, \cdots, x_n) \\ \vdots \\ f_m(x_1, x_2, \cdots, x_n) \end{pmatrix}$$

Exemple:

$$\frac{\partial f(\mathbf{x})}{\partial x_i}$$

est la **dérivée partielle** de f par rapport à la variable x_i du vecteur $\mathbf{x}$. Lors du calcul de cette dérivée, les autres variables x_j $j \neq i$ sont considérées comme constantes.

Optimisation non-linéaire

Soit une fonction f à plusieurs variables de $\mathbb{R}^n \mapsto \mathbb{R}^m$:

$$f : \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \longmapsto \begin{pmatrix} f_1(x_1, x_2, \cdots, x_n) \\ f_2(x_1, x_2, \cdots, x_n) \\ \vdots \\ f_m(x_1, x_2, \cdots, x_n) \end{pmatrix}$$

on peut dériver le vecteur ou chacune de ses composantes

$$\frac{\partial f(\mathbf{x})}{\partial x_i} = \begin{pmatrix} \partial f_1(\mathbf{x}) / \partial x_i \\ \vdots \\ \partial f_m(\mathbf{x}) / \partial x_i \end{pmatrix}$$

Matrice jacobienne

Soit une fonction f à plusieurs variables de $\mathbb{R}^n \mapsto \mathbb{R}^m$:

$$f : \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \longmapsto \begin{pmatrix} f_1(x_1, x_2, \cdots, x_n) \\ f_2(x_1, x_2, \cdots, x_n) \\ \vdots \\ f_m(x_1, x_2, \cdots, x_n) \end{pmatrix}$$

Définition:
La matrice jacobienne $\mathbf{J}$ est la matrice des dérivées partielles du premier ordre d'une fonction vectorielle f lorsqu'elles existent.

$$\mathbf{J} = \frac{\partial (f_1, \ldots, f_m)}{\partial (x_1, \ldots, x_n)} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Optimisation non-linéaire

Dérivée 1D :

$$f(x + \Delta x) \simeq f(x) + f'(x)\Delta x$$

Dérivée n-dimensionnelle :

$$f(\mathbf{x} + \Delta_\mathbf{x}) \simeq f(\mathbf{x}) + \mathbf{J}\Delta_\mathbf{x}$$

Méthode itérative

$$f(\mathbf{x} + \Delta_\mathbf{x}) \simeq f(\mathbf{x}) + \mathbf{J}\Delta_\mathbf{x}$$

Principe :
on cherche $\Delta_\mathbf{x}$ tel que $f(\mathbf{x} + \Delta_\mathbf{x}) - \widehat{\mathbf{z}} = \mathbf{0}$
$\rightarrow \underbrace{f(\mathbf{x}) - \widehat{\mathbf{z}}}_{\epsilon} + \mathbf{J}\Delta_\mathbf{x} = \mathbf{0}$, soit $\epsilon + \mathbf{J}\Delta_\mathbf{x} = \mathbf{0}$

Méthode :

- $\rightarrow$ on veut minimiser : $\mathbf{J}\Delta_\mathbf{x} = -\epsilon$
- $\rightarrow$ la solution : $\Delta_\mathbf{x} = -\mathbf{J}^+\epsilon$ $\qquad \mathbf{J}^+ = (\mathbf{J}^\top\mathbf{J})^{-1}\mathbf{J}^\top$
- $\rightarrow$ on avance : $\mathbf{x}_{i+1} = \mathbf{x}_i + \Delta_\mathbf{x}$
- $\rightarrow$ on recommence jusqu'à convergence

Méthode itérative

Algorithm 6: Newton n-dimensions

input: fonction f, solution initiale $\mathbf{x}_0$, $\widehat{\mathbf{z}}$ et un *seuil*

```
1 x = x_0
2 repeat
3 |   J = [∂f_i/∂x_j]
4 |   ε = f(x) − ẑ
5 |   Δ_x = −J⁺ε
6 |   x = x + Δ_x
7 until |f(x) − ẑ| < seuil
8 return x
```


Retour à la dimension 1

Remarque :
Il s'agit en fait de la méthode de Newton :

$$\begin{aligned} \mathbf{x}_{i+1} &= \mathbf{x}_i + \Delta_\mathbf{x} \\ &= \mathbf{x}_i - \mathbf{J}^+\epsilon \\ &= \mathbf{x}_i - \mathbf{J}^+(f(\mathbf{x}_i) - \widehat{\mathbf{z}}) \end{aligned}$$

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \qquad \longleftrightarrow \qquad \mathbf{x}_{i+1} = \mathbf{x}_i - \mathbf{J}^+(f(\mathbf{x}_i) - \widehat{\mathbf{z}})$$

En dimension 1: $\mathbf{J} = f'(x)$ est une matrice 1×1, $\mathbf{J}^+ = 1/f'(x)$.

Levenberg-Marquardt

Introduction :

Il s'agit d'une amélioration de la méthode de Newton où le pas $\Delta_{\mathbf{x}}$ a une importance variable selon la situation.

Levenberg-Marquardt

Newton :

$$\begin{array}{rcl} \mathbf{J}\Delta_{\mathbf{x}} = -\epsilon & \longrightarrow & \Delta_{\mathbf{x}} = -\mathbf{J}^{+}\epsilon \\ & \longrightarrow & \Delta_{\mathbf{x}} = -(\mathbf{J}^{\top}\mathbf{J})^{-1}\mathbf{J}^{\top}\epsilon \end{array}$$

$\Delta_{\mathbf{x}}$ est un pas acceptable si $|f(\mathbf{x}+\Delta_{\mathbf{x}})| < |f(\mathbf{x})|$, ce qui n'est pas toujours le cas.

$\Delta_{\mathbf{x}}$ trop petit :

- résolution trop longue
- risque de tomber dans un minimum local

$\Delta_{\mathbf{x}}$ trop grand :

- risque de dépasser le minimum global et de diverger

Levenberg-Marquardt

Newton :

$$\begin{array}{rcl} \mathbf{J}\Delta_{\mathbf{x}} = -\epsilon & \longrightarrow & \Delta_{\mathbf{x}} = -\mathbf{J}^{+}\epsilon \\ & \longrightarrow & \Delta_{\mathbf{x}} = -(\mathbf{J}^{\top}\mathbf{J})^{-1}\mathbf{J}^{\top}\epsilon \end{array}$$

Levenberg-Marquardt :

$$\begin{array}{rcl} \mathbf{J}\Delta_{\mathbf{x}} = -\epsilon & \longrightarrow & \mathbf{J}^{\top}\mathbf{J}\Delta_{\mathbf{x}} = -\mathbf{J}^{\top}\epsilon \\ & \longrightarrow & (\mathbf{J}^{\top}\mathbf{J} + \lambda\mathbf{Id})\Delta_{\mathbf{x}} = -\mathbf{J}^{\top}\epsilon \\ & \longrightarrow & \Delta_{\mathbf{x}} = -(\mathbf{J}^{\top}\mathbf{J} + \lambda\mathbf{Id})^{-1}\mathbf{J}^{\top}\epsilon \end{array}$$

- $\Delta_{\mathbf{x}}$ trop petit $\rightarrow$ on diminue λ
- $\Delta_{\mathbf{x}}$ trop grand $\rightarrow$ on augmente λ

Levenberg-Marquardt

En pratique :

On choisit d'abord une valeur initiale de λ_0 :

$$\lambda_0 = 10^{-3} \times \frac{\sum_{i=1}^{n}(\mathbf{J}\mathbf{J}^{\top})_{ii}}{n}$$

- si $\Delta_{\mathbf{x}}$ fait converger f : $\lambda = \lambda \div 10$
  pour accélérer la convergence
- si $\Delta_{\mathbf{x}}$ ne fait pas converger f : $\lambda = 10 \times \lambda$
  $\Delta_{\mathbf{x}}$ était trop grand

Algorithm 7: Levenberg-Marquardt

input: fonction f, solution initiale $\mathbf{x}_0$, $\widehat{\mathbf{z}}$ et un *seuil*

$\mathbf{x} = \mathbf{x}_0$
$\lambda = \frac{1}{n}\sum_{i=1}^{n}(\mathbf{J}\mathbf{J}^\top)_{ii} \times 10^{-3}$
repeat
  $\mathbf{J} = [\partial f_i/\partial x_j]$
  compteur $= 0$
  accepté $=$ **false**
  repeat
    $\Delta_{\mathbf{x}} = -(\mathbf{J}^\top\mathbf{J} + \lambda\mathbf{Id})^+\mathbf{J}^\top(f(\mathbf{x}) - \widehat{\mathbf{z}})$
    if $|f(\mathbf{x} + \Delta_{\mathbf{x}})| < |f(\mathbf{x})|$ **then**
      $\mathbf{x} = \mathbf{x} + \Delta_{\mathbf{x}}$
      $\lambda = \lambda/10$
      accepté $=$ **true**
    else $\lambda = \lambda \times 10$
    compteur $=$ *compteur* $+ 1$
    if *compteur* > 100 **then return** $\mathbf{x}$
  until *accepté* $=$ **true**
until $|f(\mathbf{x}) - \widehat{\mathbf{z}}| <$ *seuil*
return $\mathbf{x}$

Applications

Rectification d'images :




Applications

Rectification d'images :





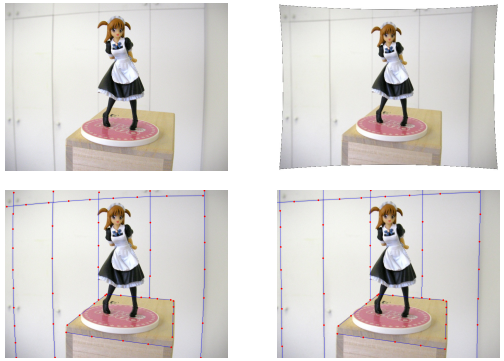

Applications

Correction de la distorsion radiale : (certains modèles sont linéaires)



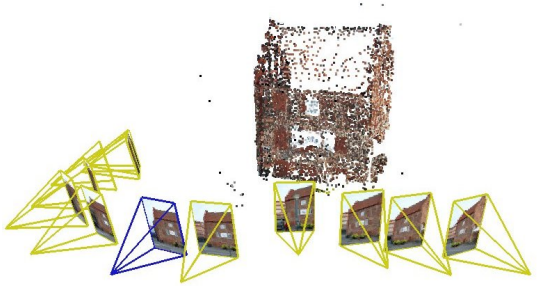

Applications

Correction de la distorsion radiale : (certains modèles sont linéaires)



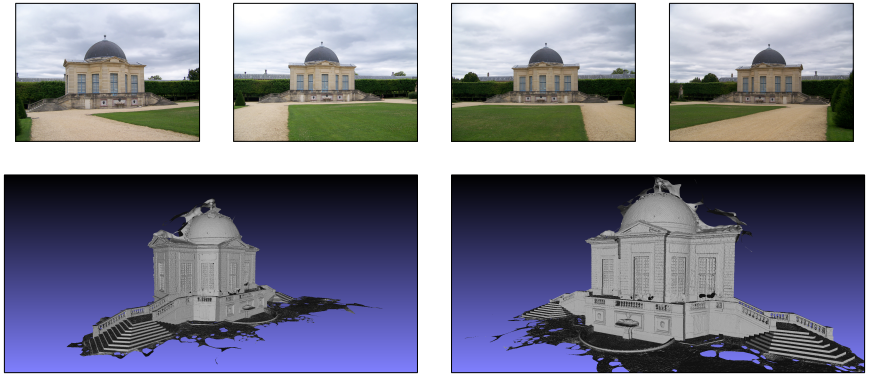
Applications

Bundle adjustment :



Applications

Bundle adjustment :



Applications

Trajectoire optimale : (courbe minimale vs. chemin le plus court)

